

Th Goodrich & Tamassia,
Algorithm design

Part

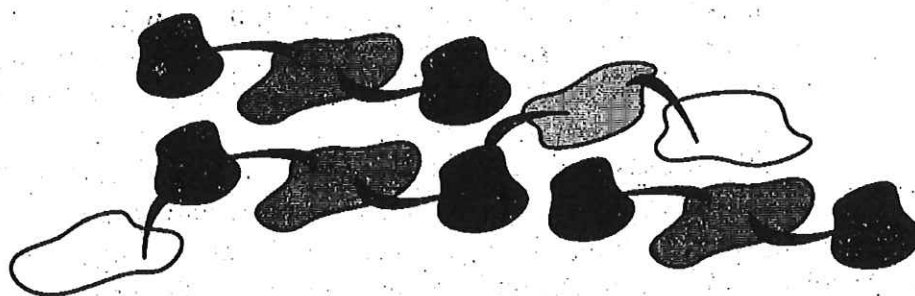
III **Internet Algorithmics**



Chapter

9

Text Processing



Contents

9.1	Strings and Pattern Matching Algorithms	419
9.1.1	String Operations	419
9.1.2	Brute Force Pattern Matching	420
9.1.3	The Boyer-Moore Algorithm	422
9.1.4	The Knuth-Morris-Pratt Algorithm	425
9.2	Tries	429
9.2.1	Standard Tries	429
9.2.2	Compressed Tries	433
9.2.3	Suffix Tries	435
9.2.4	Search Engines	439
9.3	Text Compression	440
9.3.1	The Huffman Coding Algorithm	441
9.3.2	The Greedy Method Revisited	442
9.4	Text Similarity Testing	443
9.4.1	The Longest Common Subsequence Problem	443
9.4.2	Applying Dynamic Programming to the LCS Problem	444
9.5	Exercises	447

Document processing is rapidly becoming one of the dominant functions of computers. Computers are used to edit documents, to search documents, to transport documents over the Internet, and to display documents on printers and computer screens. Web “surfing” and Web searching are becoming significant and important computer applications, and many of the key computations in all of these document processing involve character strings and string pattern matching. For example, the Internet document formats HTML and XML are primarily text formats with added links to multimedia content. Making sense of the many terabytes of information on the Internet requires a considerable amount of text processing.

In this chapter, we study several fundamental text processing algorithms for quickly performing important string operations. We pay particular attention to algorithms for string searching and pattern matching, since these can often be computational bottlenecks in many document-processing applications. We also study some fundamental algorithmic issues involved in text processing, as well.

Text processing algorithms operate primarily on character strings. The terminology and notation for strings, as used in this chapter, is fairly intuitive, and it turns out that representing a string as an array of characters is quite simple and efficient. So we don’t spend a lot of attention on string representations. Nevertheless, string processing often involves an interesting method for string pattern matching, and we study pattern matching algorithms in Section 9.1.

In Section 9.2, we study the trie data structure, which is a tree-based structure that allows for fast searching in a collection of strings.

We study an important text processing problem in Section 9.3, namely, the problem of compressing a document of text so that it fits more efficiently in storage or can be transmitted more efficiently over a network.

The final text processing problem we study, in Section 9.4, deals with how we can measure the similarity between two documents. All of these problems are topics that arise often in Internet computations, such as Web crawlers, search engines, document distribution, and information retrieval.

In addition to having interesting applications, the topics of this chapter also highlight some important algorithmic design patterns (see Chapter 5). In particular, in the section on pattern matching, we discuss the *brute-force method*, which is often inefficient but has wide applicability. For text compression we study an application of the *greedy method* (Section 5.1), which often allows us to approximate solutions to hard problems, and for some problems such as our text compression application actually gives rise to optimal algorithms. Finally, in discussing text similarity, we give another application of *dynamic programming* (Section 5.3), which can be applied, in some special instances, to solve a problem in polynomial time that appears at first to require exponential time.

9.1 Strings and Pattern Matching Algorithms

Text documents are ubiquitous in modern computing, as they are used to communicate and publish information. From the perspective of algorithm design, such documents can be viewed as simple character strings. That is, they can be abstracted as a sequence of the characters that make up their content. Performing interesting searching and processing operations on such data, therefore, requires that we have efficient methods for dealing with character strings.

9.1.1 String Operations

At the heart of algorithms for processing text are methods for dealing with character strings. Character strings can come from a wide variety of sources, including scientific, linguistic, and Internet applications. Indeed, the following are examples of such strings:

$P = \text{"CGTAAACTGCTTTAATCAAACGC"}$

$R = \text{"U.S. Men Win Soccer World Cup!"}$

$S = \text{"http://www.wiley.com/college/goodrich/"}$.

The first string, P , comes from DNA applications, the last string, S , is the Internet address (URL) for the Web site that accompanies this book, and the middle string, R , is a fictional news headline. In this section, we present some of the useful operations that are supported by the string ADT for processing strings such as these.

Several of the typical string processing operations involve breaking large strings into smaller strings. In order to be able to speak about the pieces that result from such operations, we use the term **substring** of an m -character string P to refer to a string of the form $P[i]P[i+1]P[i+2]\cdots P[j]$, for some $0 \leq i \leq j \leq m-1$, that is, the string formed by the characters in P from index i to index j , inclusive. Technically, this means that a string is actually a substring of itself (taking $i = 0$ and $j = m-1$), so if we want to rule this out as a possibility, we must restrict the definition to **proper** substrings, which require that either $i > 0$ or $j < m-1$. To simplify the notation for referring to substrings, let us use $P[i..j]$ to denote the substring of P from index i to index j , inclusive. That is,

$$P[i..j] = P[i]P[i+1]\cdots P[j].$$

We use the convention that if $i > j$, then $P[i..j]$ is equal to the **null string**, which has length 0. In addition, in order to distinguish some special kinds of substrings, let us refer to any substring of the form $P[0..i]$, for $0 \leq i \leq m-1$, as a **prefix** of P , and any substring of the form $P[i..m-1]$, for $0 \leq i \leq m-1$, as a **suffix** of P . For example, if we again take P to be the string of DNA given above, then "CGTAA" is a prefix of P , "CGC" is a suffix of P , and "TTAATC" is a (proper) substring of P . Note that the null string is a prefix and a suffix of any other string.

The Pattern Matching Problem

In the classic *pattern matching* problem on strings, we are given a *text* string T of length n and a *pattern* string P of length m , and want to find whether P is a substring of T . The notion of a “match” is that there is a substring of T starting at some index i that matches P , character by character, so that

$$T[i] = P[0], T[i+1] = P[1], \dots, T[i+m-1] = P[m-1].$$

That is,

$$P = T[i..i+m-1].$$

Thus, the output from a pattern matching algorithm is either an indication that the pattern P does not exist in T or the starting index in T of a substring matching P .

To allow for fairly general notions of a character string, we typically do not restrict the characters in T and P to come explicitly from a well-known character set, like the ASCII or Unicode character sets. Instead, we typically use the general symbol Σ to denote the character set, or *alphabet*, from which the characters in T and P can come. This alphabet Σ can, of course, be a subset of the ASCII or Unicode character sets, but it could also be more general and is even allowed to be infinite. Nevertheless, since most document processing algorithms are used in applications where the underlying character set is finite, we usually assume that the size of the alphabet Σ , denoted with $|\Sigma|$, is a fixed constant.

Example 9.1: Suppose we are given the text string

$$T = \text{"abacaabaccabacabaabb"}$$

and the pattern string

$$P = \text{"abacab"}.$$

Then P is a substring of T . Namely, $P = T[10..15]$.

In this section, we present three different pattern matching algorithms.

9.1.2 Brute Force Pattern Matching

The *brute force* algorithmic design pattern is a powerful technique for algorithm design when we have something we wish to search for or when we wish to optimize some function. In applying this technique in a general situation, we typically enumerate all possible configurations of the inputs involved and pick the best of all these enumerated configurations.

Brute-Force Pattern Matching

In applying this technique to design the *brute-force pattern matching* algorithm, we derive what is probably the first algorithm that we might think of for solving the pattern matching problem—we simply test all the possible placements of P relative to T . This approach, shown in Algorithm 9.1, is quite simple.

Algorithm BruteForceMatch(T, P):

Input: Strings T (text) with n characters and P (pattern) with m characters

Output: Starting index of the first substring of T matching P , or an indication that P is not a substring of T

for $i \leftarrow 0$ **to** $n - m$ {for each candidate index in T } **do**

$j \leftarrow 0$

while ($j < m$ and $T[i + j] = P[j]$) **do**

$j \leftarrow j + 1$

if $j = m$ **then**

return i

return "There is no substring of T matching P ."

Algorithm 9.1: Brute-force pattern matching.

The brute-force pattern matching algorithm could not be simpler. It consists of two nested loops, with the outer loop indexing through all possible starting indices of the pattern in the text, and the inner loop indexing through each character of the pattern, comparing it to its potentially corresponding character in the text. Thus, the correctness of the brute-force pattern matching algorithm follows immediately.

The running time of brute-force pattern matching in the worst case is not good, however, because, for each candidate index in T , we can perform up to m character comparisons to discover that P does not match T at the current index. Referring to Algorithm 9.1, we see that the outer for-loop is executed at most $n - m + 1$ times, and the inner loop is executed at most m times. Thus, the running time of the brute-force method is $O((n - m + 1)m)$, which is $O(nm)$. Thus, in the worst case, when n and m are roughly equal, this algorithm has a quadratic running time.

In Figure 9.2 we illustrate the execution of the brute-force pattern matching algorithm on the strings T and P from Example 9.1.

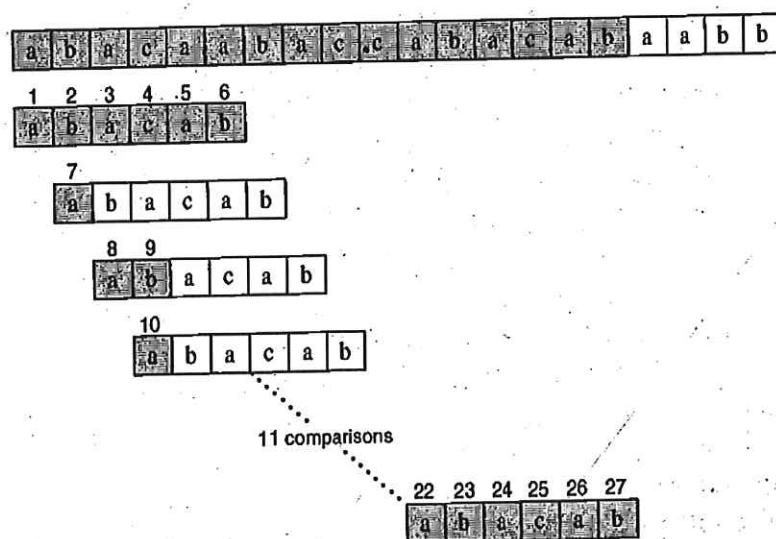


Figure 9.2: Example run of the brute-force pattern matching algorithm. The algorithm performs 27 character comparisons, indicated above with numerical labels.

9.1.3 The Boyer-Moore Algorithm

At first, we might feel that it is always necessary to examine every character in T in order to locate a pattern P as a substring. But this is not always the case for the *Boyer-Moore (BM)* pattern matching algorithm, which we study in this section, can sometimes avoid comparisons between P and a sizable fraction of the characters in T . The only caveat is that, whereas the brute-force algorithm can work even with a potentially unbounded alphabet, the BM algorithm assumes the alphabet is of fixed, finite size. It works the fastest when the alphabet is moderately sized and the pattern is relatively long.

In this section, we describe a simplified version of the original BM algorithm. The main idea is to improve the running time of the brute-force algorithm by adding two potentially time-saving heuristics:

Looking-Glass Heuristic: When testing a possible placement of P against T , begin the comparisons from the end of P and move backward to the front of P .

Character-Jump Heuristic: During the testing of a possible placement of P against T , a mismatch of text character $T[i] = c$ with the corresponding pattern character $P[j]$ is handled as follows. If c is not contained anywhere in P , then shift P completely past $T[i]$ (for it cannot match any character in P). Otherwise, shift P until an occurrence of character c in P gets aligned with $T[i]$.

We will formalize these heuristics shortly, but at an intuitive level, they work as an integrated team. The looking-glass heuristic sets up the other heuristic to allow us to avoid comparisons between P and whole groups of characters in T . In this case at least, we can get to the destination faster by going backwards, for if we encounter a mismatch during the consideration of P at a certain location in T , then we are likely to avoid lots of needless comparisons by significantly shifting P relative to T using the character-jump heuristic. The character-jump heuristic pays off big if it can be applied early in the testing of a potential placement of P against T .

Therefore, let us define how the character-jump heuristics can be integrated into a string pattern matching algorithm. To implement this heuristic, we define a function $\text{last}(c)$ that takes a character c from the alphabet and specifies how far we may shift the pattern P if a character equal to c is found in the text that does not match the pattern. In particular, we define $\text{last}(c)$ as follows:

- If c is in P , $\text{last}(c)$ is the index of the last (right-most) occurrence of c in P . Otherwise, we conventionally define $\text{last}(c) = -1$.

If characters can be used as indices in arrays, then the last function can be easily implemented as a lookup table. We leave the method for computing this table efficiently as a simple exercise (R-9.6). The last function will give us all the information we need to perform the character-jump heuristic. In Algorithm 9.3, we show the BM pattern matching method. The jump step is illustrated in Figure 9.4.

Algorithm BMMatch(T, P):

Input: Strings T (text) with n characters and P (pattern) with m characters

Output: Starting index of the first substring of T matching P , or an indication that P is not a substring of T

compute function last

$i \leftarrow m - 1$

$j \leftarrow m - 1$

repeat

if $P[j] = T[i]$ **then**

if $j = 0$ **then**

return i { a match! }

else

$i \leftarrow i - 1$

$j \leftarrow j - 1$

else

$i \leftarrow i + m - \min(j, 1 + \text{last}(T[i]))$ { jump step }

$j \leftarrow m - 1$

until $i > n - 1$

return "There is no substring of T matching P ."

Algorithm 9.3: The Boyer-Moore pattern matching algorithm.

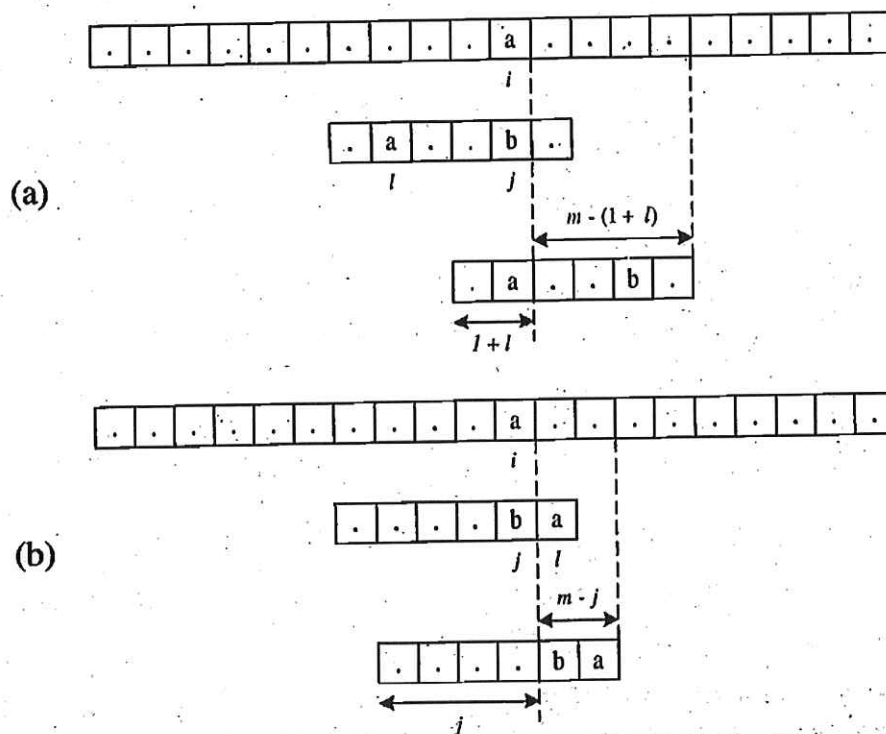
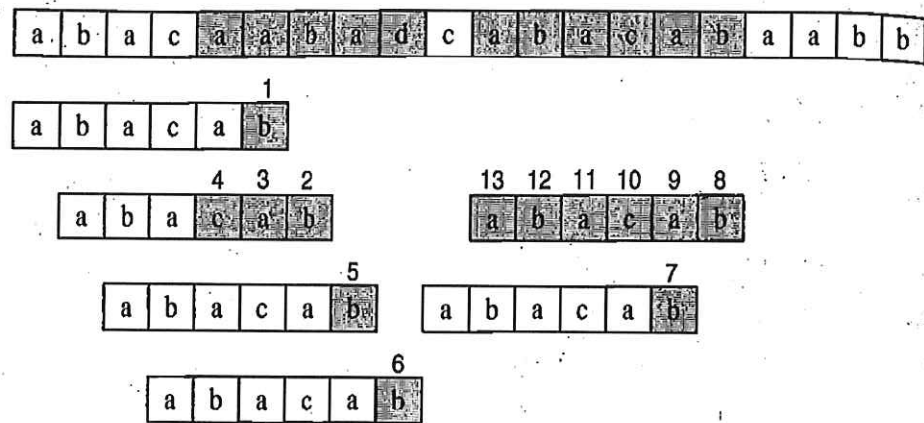


Figure 9.4: Illustration of the jump step in the BM algorithm, where l denotes $\text{last}(T[i])$. We distinguish two cases: (a) $1 + l \leq j$, where we shift the pattern by $j - l$ units; (b) $j < 1 + l$, where we shift the pattern by one unit.

In Figure 9.5, we illustrate the execution of the Boyer-Moore pattern matching algorithm on a similar input string as in Example 9.1.



The last(c) function:

c	a	b	c	d
last(c)	4	5	3	-1

Figure 9.5: An illustration of the BM pattern matching algorithm. The algorithm performs 13 character comparisons, which are indicated with numerical labels.

The correctness of the BM pattern matching algorithm follows from the fact that each time the method makes a shift, it is guaranteed not to “skip” over any possible matches. For last(c) is the location of the *last* occurrence of c in P .

The worst-case running time of the BM algorithm is $O(nm + |\Sigma|)$. Namely, the computation of the last function takes time $O(m + |\Sigma|)$ and the actual search for the pattern takes $O(nm)$ time in the worst case, the same as the brute-force algorithm. An example of a text-pattern pair that achieves the worst case is

$$T = \underbrace{aaaaaa \cdots a}_n$$

$$P = b \underbrace{aa \cdots a}_{m-1}.$$

The worst-case performance, however, is unlikely to be achieved for English text.

Indeed, the BM algorithm is often able to skip over large portions of the text. (See Figure 9.6.) There is experimental evidence that on English text, the average number of comparisons done per text character is approximately 0.24 for a five-character pattern string. The payoff is not as great for binary strings or for very short patterns, however, in which case the KMP algorithm, discussed in Section 9.1.4, or for very short patterns, the brute-force algorithm, may be better.

Example 9.2: Consider the pattern string $P = \text{"abacab"}$ from Example 9.1. The KMP failure function $f(j)$ for the string P is as shown in the following table.

j	0	1	2	3	4	5
$P[j]$	a	b	a	c	a	b
$f(j)$	0	0	1	0	1	2

The KMP pattern matching algorithm, shown in Algorithm 9.7, incrementally processes the text string T comparing it to the pattern string P . Each time there is a match, we increment the current indices. On the other hand, if there is a mismatch and we have previously made progress in P , then we consult the failure function to determine the new index in P where we need to continue checking P against T . Otherwise (there was a mismatch and we are at the beginning of P), we simply increment the index for T (and keep the index variable for P at its beginning). We repeat this process until we find a match of P in T or the index for T reaches the length of T (indicating that we did not find the pattern P in T).

The main part of the KMP algorithm is the while-loop, which performs a comparison between a character in T and a character in P each iteration. Depending upon the outcome of this comparison, the algorithm either moves on to the next characters in T and P , consults the failure function for a new candidate character in P , or starts over with the next index in T . The correctness of this algorithm follows from the definition of the failure function. The skipped comparisons are actually unnecessary, for the failure function guarantees that all the ignored comparisons are redundant—they would involve comparing characters we already know match.

Algorithm KMPMatch(T, P):

Input: Strings T (text) with n characters and P (pattern) with m characters

Output: Starting index of the first substring of T matching P , or an indication that P is not a substring of T

$f \leftarrow \text{KMPPailureFunction}(P)$ {construct the failure function f for P }

$i \leftarrow 0$

$j \leftarrow 0$

while $i < n$ **do**

if $P[j] = T[i]$ **then**

if $j = m - 1$ **then**

return $i - m + 1$ {a match!}

$i \leftarrow i + 1$

$j \leftarrow j + 1$

else if $j > 0$ {no match, but we have advanced in P } **then**

$j \leftarrow f(j - 1)$ { j indexes just after prefix of P that must match}

else

$i \leftarrow i + 1$

return "There is no substring of T matching P ."

Algorithm 9.7: The KMP pattern matching algorithm.

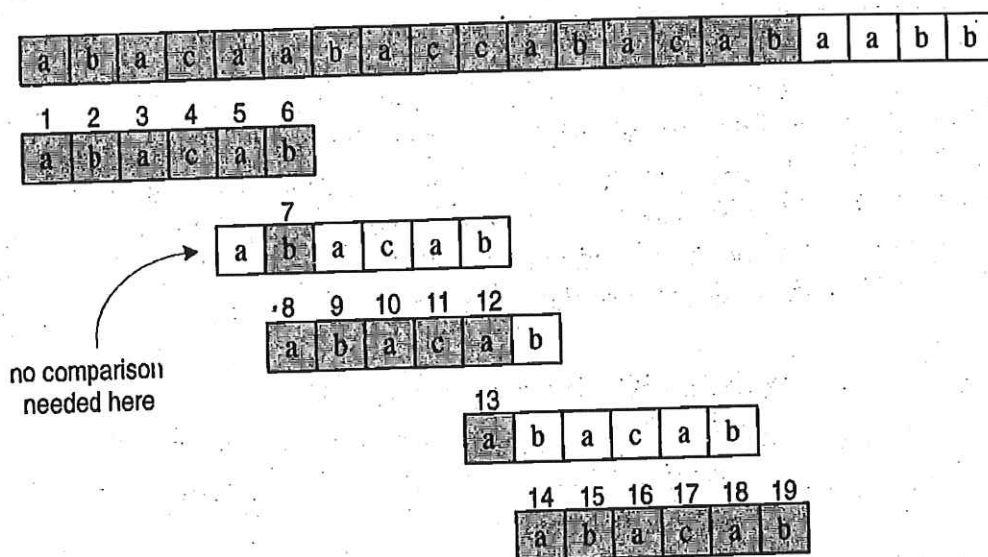


Figure 9.8: An illustration of the KMP pattern matching algorithm. The failure function f for this pattern is given in Example 9.2. The algorithm performs 19 character comparisons, which are indicated with numerical labels.

In Figure 9.8, we illustrate the execution of the KMP pattern matching algorithm on the same input strings as in Example 9.1. Note the use of the failure function to avoid redoing one of the comparisons between a character of the pattern and a character of the text. Also note that the algorithm performs fewer overall comparisons than the brute-force algorithm run on the same strings (Figure 9.2).

Performance

Excluding the computation of the failure function, the running time of the KMP algorithm is clearly proportional to the number of iterations of the while-loop. For the sake of the analysis, let us define $k = i - j$. Intuitively, k is the total amount by which the pattern P has been shifted with respect to the text T . Note that throughout the execution of the algorithm, we have $k \leq n$. One of the following three cases occurs at each iteration of the loop.

- If $T[i] = P[j]$, then i increases by 1, and k does not change, since j also increases by 1.
- If $T[i] \neq P[j]$ and $j > 0$, then i does not change and k increases by at least 1, since in this case k changes from $i - j$ to $i - f(j - 1)$, which is an addition of $j - f(j - 1)$, which is positive because $f(j - 1) < j$.
- If $T[i] \neq P[j]$ and $j = 0$, then i increases by 1 and k increases by 1, since j does not change.

Thus, at each iteration of the loop, either i or k increases by at least 1 (possibly both); hence, the total number of iterations of the while-loop in the KMP pattern matching algorithm is at most $2n$. Of course, achieving this bound assumes that we have already computed the failure function for P .

Constructing the KMP Failure Function

To construct the failure function used in the KMP pattern matching algorithm, we use the method shown in Algorithm 9.9. This algorithm is another example of a “bootstrapping” process quite similar to that used in the KMPMatch algorithm. We compare the pattern to itself as in the KMP algorithm. Each time we have $i > j$ characters that match, we set $f(i) = j + 1$. Note that since we have $i > j$ throughout the execution of the algorithm, $f(j - 1)$ is always defined when we need to

Algorithm KMPFailureFunction(P):

Input: String P (pattern) with m characters

Output: The failure function f for P , which maps j to the length of the longest prefix of P that is a suffix of $P[1..j]$

```

 $i \leftarrow 1$ 
 $j \leftarrow 0$ 
 $f(0) \leftarrow 0$ 
while  $i < m$  do
  if  $P[j] = P[i]$  then
    { we have matched  $j + 1$  characters }
     $f(i) \leftarrow j + 1$ 
     $i \leftarrow i + 1$ 
     $j \leftarrow j + 1$ 
  else if  $j > 0$  then
    {  $j$  indexes just after a prefix of  $P$  that must match }
     $j \leftarrow f(j - 1)$ 
  else
    { we have no match here }
     $f(i) \leftarrow 0$ 
     $i \leftarrow i + 1$ 

```

Algorithm 9.9: Computation of the failure function used in the KMP pattern matching algorithm. Note how the algorithm uses the previous values of the failure function to efficiently compute new values.

Algorithm KMPFailureFunction runs in $O(m)$ time. Its analysis is analogous to that of algorithm KMPMatch. Thus, we have:

Theorem 9.3: The Knuth-Morris-Pratt algorithm performs pattern matching on a text string of length n and a pattern string of length m in $O(n + m)$ time.

The running time analysis of the KMP algorithm may seem a little surprising at first, for it states that, in time proportional to that needed just to read the strings T and P separately, we can find the first occurrence of P in T . Also, it should be noted that the running time of the KMP algorithm does not depend on the size of the alphabet.

Tries

The pattern matching algorithms presented in the previous section speed up the search in a text by preprocessing the pattern (to compute the failure function in the KMP algorithm or the last function in the BM algorithm). In this section, we take a complementary approach, namely, we present string searching algorithms that preprocess the text. This approach is suitable for applications where a series of queries is performed on a fixed text, so that the initial cost of preprocessing the text is compensated by a speedup in each subsequent query (for example, a Web site that offers pattern matching in Shakespeare's *Hamlet* or a search engine that offers Web pages on the *Hamlet* topic).

A *trie* (pronounced "try") is a tree-based data structure for storing strings in order to support fast pattern matching. The main application for tries is in information retrieval. Indeed, the name "trie" comes from the word "retrieval." In an information retrieval application, such as a search for a certain DNA sequence in a genomic database, we are given a collection S of strings, all defined using the same alphabet.

The primary query operations that tries support are pattern matching and *prefix matching*. The latter operation involves being given a string X , and looking for all the strings in S that contain X as a prefix.

9.2.1 Standard Tries

Let S be a set of s strings from alphabet Σ , such that no string in S is a prefix of another string. A *standard trie* for S is an ordered tree T with the following properties (see Figure 9.10):

- Each node of T , except the root, is labeled with a character of Σ .
- The ordering of the children of an internal node of T is determined by a canonical ordering of the alphabet Σ .
- T has s external nodes, each associated with a string of S , such that the concatenation of the labels of the nodes on the path from the root to an external node v of T yields the string of S associated with v .

Thus, a trie T represents the strings of S with paths from the root to the external nodes of T . Note the importance of assuming that no string in S is a prefix of another string. This ensures that each string of S is uniquely associated with an external node of T . We can always satisfy this assumption by adding a special character that is not in the original alphabet Σ at the end of each string.

An internal node in a standard trie T can have anywhere between 1 and d children, where d is the size of the alphabet. There is an edge going from the root r to one of its children for each character that is first in some string in the collection S .

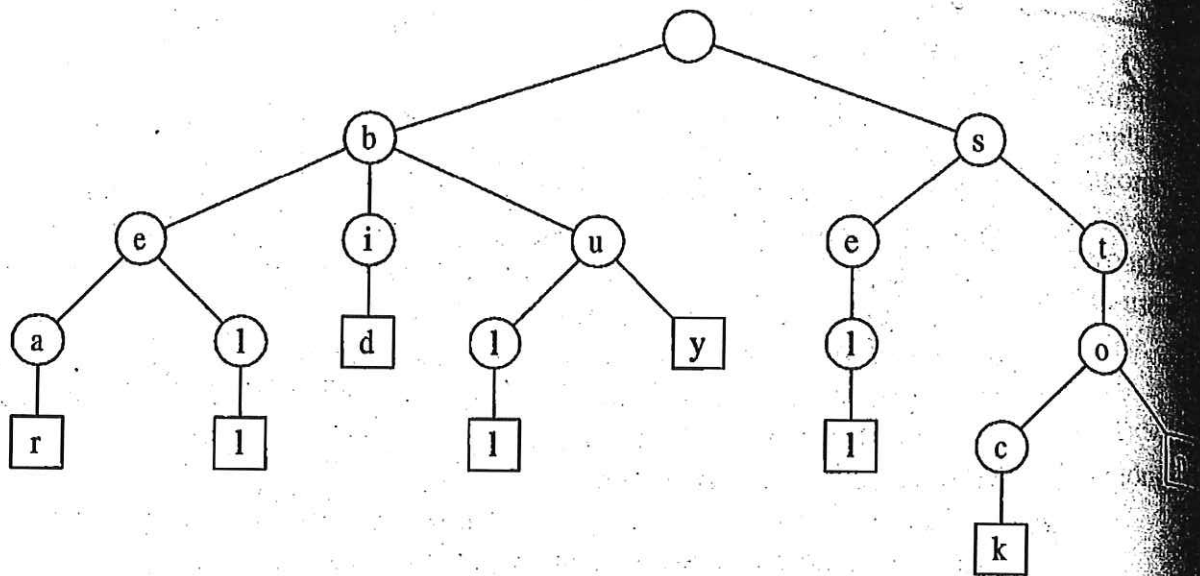


Figure 9.10: Standard trie for the strings {bear, bell, bid, bull, buy, sell, stock, stop}

In addition, a path from the root of T to an internal node v at depth i corresponds to an i -character prefix $X[0..i-1]$ of a string X of S . In fact, for each character c that can follow the prefix $X[0..i-1]$ in a string of the set S , there is a child of v labeled with character c . In this way, a trie concisely stores the common prefixes that exist among a set of strings.

If there are only two characters in the alphabet, then the trie is essentially a binary tree, although some internal nodes may have only one child (that is, it may be an improper binary tree). In general, if there are d characters in the alphabet, then the trie will be a multi-way tree where each internal node has between 1 and d children. In addition, there are likely to be several internal nodes in a standard trie that have fewer than d children. For example, the trie shown in Figure 9.10 has several internal nodes with only one child. We can implement a trie with a tree storing characters at its nodes.

The following theorem provides some important structural properties of a standard trie:

Theorem 9.4: A standard trie storing a collection S of s strings of total length n from an alphabet of size d has the following properties:

- Every internal node of T has at most d children
- T has s external nodes
- The height of T is equal to the length of the longest string in S
- The number of nodes of T is $O(n)$.

Performance

The worst case for the number of nodes of a trie occurs when no two strings share a common nonempty prefix; that is, except for the root, all internal nodes have one child.

A trie T for a set S of strings can be used to implement a dictionary whose keys are the strings of S . Namely, we perform a search in T for a string X by tracing down from the root the path indicated by the characters in X . If this path can be traced and terminates at an external node, then we know X is in the dictionary. For example, in the trie in Figure 9.10, tracing the path for “bull” ends up at an external node. If the path cannot be traced or the path can be traced but terminates at an internal node, then X is not in the dictionary. In the example in Figure 9.10, the path for “bet” cannot be traced and the path for “be” ends at an internal node. Neither such word is in the dictionary. Note that in this implementation of a dictionary, single characters are compared instead of the entire string (key).

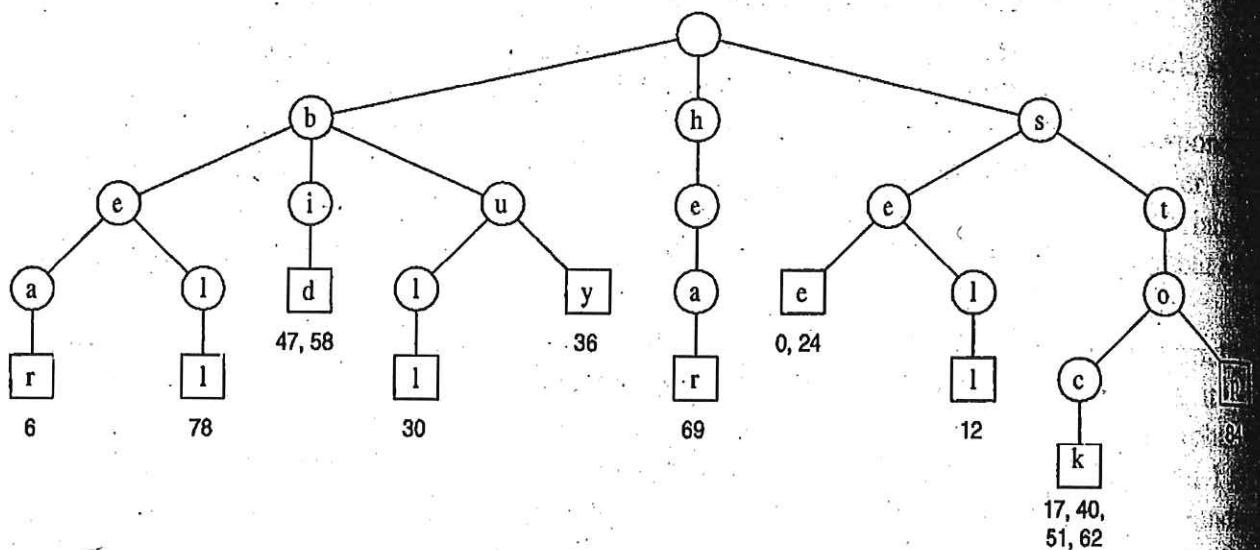
It is easy to see that the running time of the search for a string of size m is $O(dm)$, where d is the size of the alphabet. Indeed, we visit at most $m + 1$ nodes of T and we spend $O(d)$ time at each node. For some alphabets, we may be able to improve the time spent at a node to be $O(1)$ or $O(\log d)$ by using a dictionary of characters implemented in a hash table or lookup table. However, since d is a constant in most applications, we can stick with the simple approach that takes $O(d)$ time per node visited.

From the above discussion, it follows that we can use a trie to perform a special type of pattern matching, called *word matching*, where we want to determine whether a given pattern matches one of the words of the text exactly. (See Figure 9.11.) Word matching differs from standard pattern matching since the pattern cannot match an arbitrary substring of the text, but only one of its words. Using a trie, word matching for a pattern of length m takes $O(dm)$ time, where d is the size of the alphabet, independent of the size of the text. If the alphabet has constant size (as is the case for text in natural languages and DNA strings), a query takes $O(m)$ time, proportional to the size of the pattern. A simple extension of this scheme supports prefix matching queries. However, arbitrary occurrences of the pattern in the text (for example, the pattern is a proper suffix of a word or spans two words) cannot be efficiently performed.

To construct a standard trie for a set S of strings, we can use an incremental algorithm that inserts the strings one at a time. Recall the assumption that no string of S is a prefix of another string. To insert a string X into the current trie T , we first try to trace the path associated with X in T . Since X is not already in T and no string in S is a prefix of another string, we will stop tracing the path at an *internal* node v of T before reaching the end of X . We then create a new chain of node descendants of v to store the remaining characters of X . The time to insert X is $O(dm)$, where m is the length of X and d is the size of the alphabet. Thus, constructing the entire trie for set S takes $O(dn)$ time, where n is the total length of the strings of S .

s	e	e		a		b	e	a	r	?		s	e	l	l		s	t	o	c	k	!
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
s	e	e		a		b	u	l	l	?		b	u	y		s	t	o	c	k	!	
24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46
b	i	d		s	t	o	c	k	!		b	i	d		s	t	o	c	k	!		
47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	
h	e	a	r		t	h	e		b	e	l	l	?		s	t	o	p	!			
69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88			

(a)



(b)

Figure 9.11: Word matching and prefix matching with a standard trie: (a) an example text that is to be searched; (b) a standard trie for the words in the text (with articles and prepositions, which are also known as *stop words*, excluded). We show external nodes augmented with indications of the corresponding word positions.

There is a potential space inefficiency in the standard trie that has prompted the development of the *compressed trie*, which is also known (for historical reasons) as the *Patricia trie*. Namely, there are potentially a lot of nodes in the standard trie that have only one child, and the existence of such nodes is a waste, for it implies that the total number of nodes in the tree could be more than the number of words in the corresponding text.

We discuss the compressed trie data structure in the next subsection.

9.2.2 Compressed Tries

A *compressed trie* is similar to a standard trie but it ensures that each internal node in the trie has at least two children. It enforces this rule by compressing chains of single-child nodes into individual edges. (See Figure 9.12.) Let T be a standard trie. We say that an internal node v of T is *redundant* if v has one child and is not the root. For example, the trie of Figure 9.10 has eight redundant nodes. Let us also say that a chain of $k \geq 2$ edges,

$$(v_0, v_1)(v_1, v_2) \cdots (v_{k-1}, v_k),$$

is *redundant* if

- v_i is redundant for $i = 1, \dots, k-1$
- v_0 and v_k are not redundant.

We can transform T into a compressed trie by replacing each redundant chain $(v_0, v_1) \cdots (v_{k-1}, v_k)$ of $k \geq 2$ edges into a single edge (v_0, v_k) , relabeling v_k with the concatenation of the labels of nodes $v_1, \dots, v_k$.

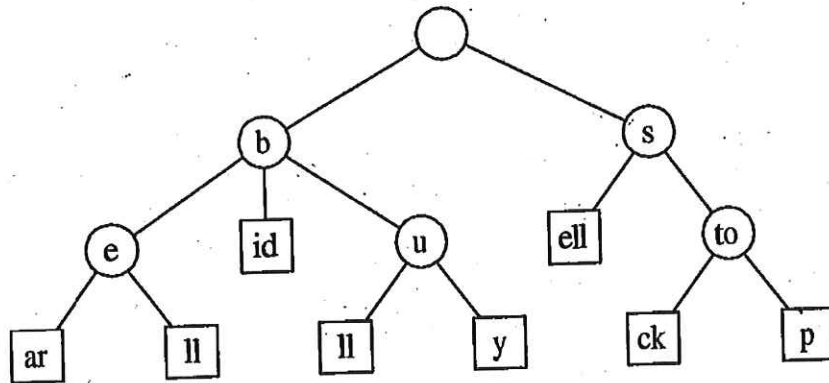


Figure 9.12: Compressed trie for the strings {bear, bell, bid, bull, buy, sell, stock, stop}. Compare this with the standard trie shown in Figure 9.10.

Thus, nodes in a compressed trie are labeled with strings, which are substrings of strings in the collection, rather than with individual characters. The advantage of a compressed trie over a standard trie is that the number of nodes of the compressed trie is proportional to the number of strings and not to their total length, as shown in the following theorem (compare with Theorem 9.4).

Theorem 9.5: A compressed trie storing a collection S of s strings from an alphabet of size d has the following properties:

- Every internal node of T has at least two children and at most d children
- T has s external nodes
- The number of nodes of T is $O(s)$.

The attentive reader may wonder whether the compression of paths provides any significant advantage, since it is offset by a corresponding expansion of node labels. Indeed, a compressed trie is truly advantageous only when it is used as an *auxiliary* index structure over a collection of strings already stored in a primary structure, and is not required to actually store all the characters of the strings in the collection. Given this auxiliary structure, however, the compressed trie is indeed quite efficient.

Suppose, for example, that the collection S of strings is an array of strings $S[0], S[1], \dots, S[s-1]$. Instead of storing the label X of a node explicitly, we represent it implicitly by a triplet of integers (i, j, k) , such that $X = S[i][j..k]$; that is, X is the substring of $S[i]$ consisting of the characters from the j th to the k th included. See the example in Figure 9.13. Also compare with the standard trie of Figure 9.11.

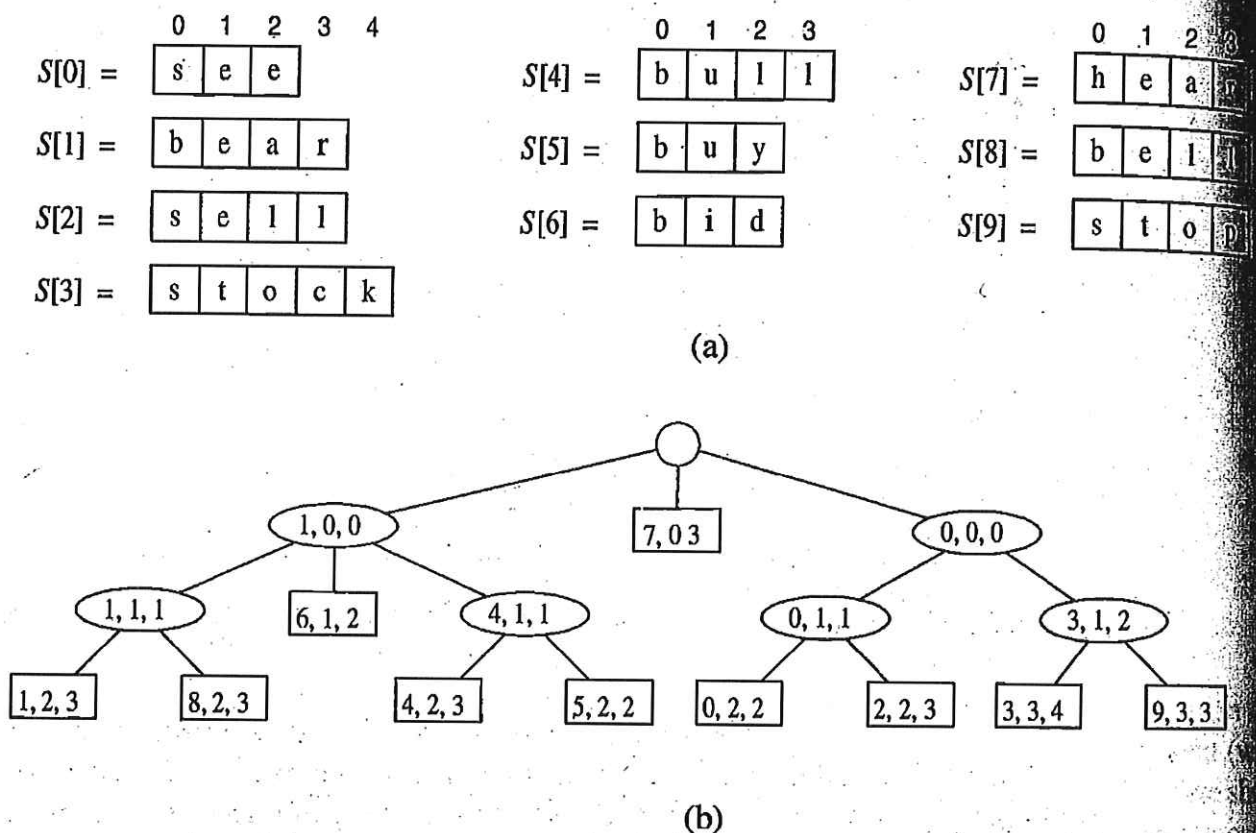


Figure 9.13: (a) Collection S of strings stored in an array. (b) Compact representation of the compressed trie for S .

This additional compression scheme allows us to reduce the total space for the trie itself from $O(n)$ for the standard trie to $O(s)$ for the compressed trie, where n is the total length of the strings in S and s is the number of strings in S . We must still store the different strings in S , of course, but we nevertheless reduce the space for the trie. In the next section, we present an application where the collection of strings can also be stored compactly.

9.2.3 Suffix Tries

One of the primary applications for tries is for the case when the strings in the collection S are all the suffixes of a string X . Such a trie is called the *suffix trie* (also known as a *suffix tree* or *position tree*) of string X . For example, Figure 9.14a shows the suffix trie for the eight suffixes of string "minimize".

For a suffix trie, the compact representation presented in the previous section can be further simplified. Namely, we can construct the trie so that the label of each vertex is a pair (i, j) indicating the string $X[i..j]$. (See Figure 9.14b.) To satisfy the rule that no suffix of X is a prefix of another suffix, we can add a special character, denoted with \$, that is not in the original alphabet Σ at the end of X (and thus to every suffix). That is, if string X has length n , we build a trie for the set of n strings $X[i..n-1]\$,$ for $i = 0, \dots, n-1$.

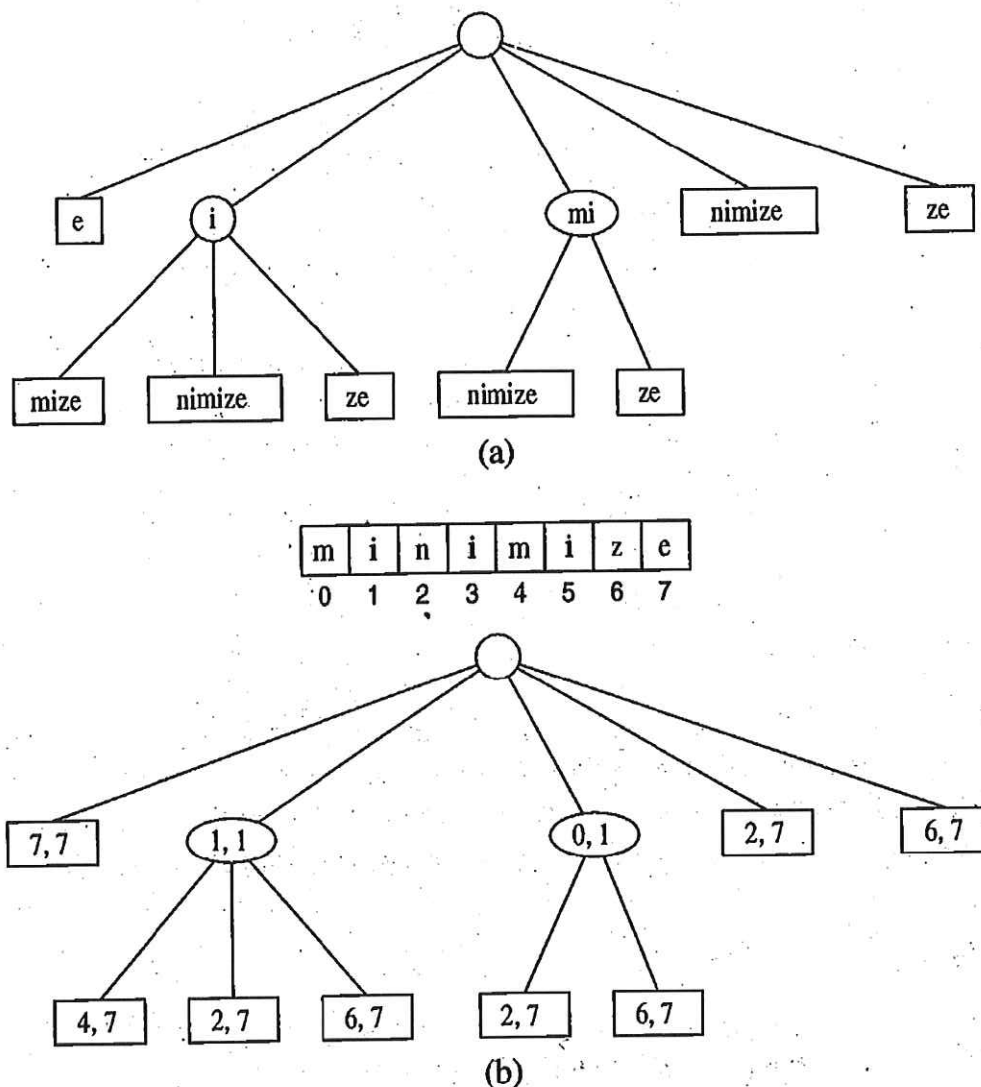


Figure 9.14: (a) Suffix trie T for the string $X = \text{"minimize"}$, (b) Compact representation of T , where pair (i, j) denotes $X[i..j]$.

Saving Space

Using a suffix trie allows us to save space over a standard trie by using several space compression techniques, including those used for the compressed trie. The advantage of the compact representation of tries now becomes apparent for suffix tries. Since the total length of the suffixes of a string X of length n is

$$1 + 2 + \dots + n = \frac{n(n+1)}{2},$$

storing all the suffixes of X explicitly would take $O(n^2)$ space. Even so, the suffix trie represents these strings implicitly in $O(n)$ space, as formally stated in the following theorem.

Theorem 9.6: *The compact representation of a suffix trie T for a string X of length n uses $O(n)$ space.*

Construction

We can construct the suffix trie for a string of length n with an incremental algorithm like the one given in Section 9.2.1. This construction takes $O(dn^2)$ time because the total length of the suffixes is quadratic in n . However, the (compact) suffix trie for a string of length n can be constructed in $O(n)$ time with a specialized algorithm, different from the one for general tries. This linear-time construction algorithm is fairly complex, however, and is not reported here. Still, we can take advantage of the existence of this fast construction algorithm when we want to use a suffix trie to solve other problems.

Using a Suffix Trie

The suffix trie T for a string X can be used to efficiently perform pattern matching queries on text X . Namely, we can determine whether a pattern P is a substring of X by trying to trace a path associated with P in T . P is a substring of X if and only if such a path can be traced. The details of the pattern matching algorithm are given in Algorithm 9.15, which assumes the following additional property on the labels of the nodes in the compact representation of the suffix trie:

If node v has label (i, j) and Y is the string of length y associated with the path from the root to v (included), then $X[j - y + 1..j] = Y$.

This property ensures that we can easily compute the start index of the pattern in the text when a match occurs.

Algorithm suffixTrieMatch(T, P):

Input: Compact suffix trie T for a text X and pattern P

Output: Starting index of a substring of X matching P or an indication that P is not a substring of X

```

 $p \leftarrow P.length()$  { length of suffix of the pattern to be matched }
 $j \leftarrow 0$  { start of suffix of the pattern to be matched }
 $v \leftarrow T.root()$ 
repeat
     $f \leftarrow \text{true}$  { flag indicating that no child was successfully processed }
    for each child  $w$  of  $v$  do
         $i \leftarrow \text{start}(v)$ 
        if  $P[j] = T[i]$  then
            { process child  $w$  }
             $x \leftarrow \text{end}(w) - i + 1$ 
            if  $p \leq x$  then
                { suffix is shorter than or of the same length of the node label }
                if  $P[j..j+p-1] = X[i..i+p-1]$  then
                    return  $i - j$  { match }
            else
                return "P is not a substring of X"
        else
            { suffix is longer than the node label }
            if  $P[j..j+x-1] = X[i..i+x-1]$  then
                 $p \leftarrow p - x$  { update suffix length }
                 $j \leftarrow j + x$  { update suffix start index }
                 $v \leftarrow w$ 
                 $f \leftarrow \text{false}$ 
            break out of the for loop
until  $f$  or  $T.isExternal(v)$ 
return "P is not a substring of X"

```

Algorithm 9.15: Pattern matching with a suffix trie. We denote the label of a node v with $(\text{start}(v), \text{end}(v))$, that is, the pair of indices specifying the substring of the text associated with v .

Suffix Trie Properties

The correctness of algorithm `suffixTrieMatch` follows from the fact that we search down the trie T , matching characters of the pattern P one at a time until one of the following events occurs:

- We completely match the pattern P
- We get a mismatch (caught by the termination of the for-loop without break-out)
- We are left with characters of P still to be matched after processing an external node.

Let m be the size of pattern P and d be the size of the alphabet. In order to determine the running time of algorithm `suffixTrieMatch`, we make the following observations:

- We process at most $m + 1$ nodes of the trie
- Each node processed has at most d children
- At each node v processed, we perform at most one character comparison for each child w of v to determine which child of v needs to be processed next (which may possibly be improved by using a fast dictionary to index the children of v)
- We perform at most m character comparisons overall in the processed nodes
- We spend $O(1)$ time for each character comparison.

Performance

We conclude that algorithm `suffixTrieMatch` performs pattern matching queries in $O(dm)$ time (and would possibly run even faster if we used a dictionary to index children of nodes in the suffix trie). Note that the running time does not depend on the size of the text X . Also, the running time is linear in the size of the pattern; that is, it is $O(m)$, for a constant-size alphabet. Hence, suffix tries are suited for repetitive pattern matching applications, where a series of pattern matching queries is performed on a fixed text.

We summarize the results of this section in the following theorem.

Theorem 9.7: *Let X be a text string with n characters from an alphabet of size d . We can perform pattern matching queries on X in $O(dm)$ time, where m is the length of the pattern, with the suffix trie of X , which uses $O(n)$ space and can be constructed in $O(dn)$ time.*

We explore another application of tries in the next subsection.

9.2.4 Search Engines

The World Wide Web contains a huge collection of text documents (Web pages). Information about these pages is gathered by a program called a *Web crawler*, which then stores this information in a special dictionary database. A *Web search engine* allows users to retrieve relevant information from this database, thereby identifying relevant pages on the Web containing given keywords. In this section, we present a simplified model of a search engine.

Inverted Files

The core information stored by a search engine is a dictionary, called an *inverted index* or *inverted file*, storing key-value pairs (w, L) , where w is a word and L is a collection of references to pages containing word w . The keys (words) in this dictionary are called *index terms* and should be a set of vocabulary entries and proper nouns as large as possible. The elements in this dictionary are called *occurrence lists* and should cover as many Web pages as possible.

We can efficiently implement an inverted index with a data structure consisting of the following:

- An array storing the occurrence lists of the terms (in no particular order)
- A compressed trie for the set of index terms, where each external node stores the index of the occurrence list of the associated term.

The reason for storing the occurrence lists outside the trie is to keep the size of the trie data structure sufficiently small to fit in internal memory. Instead, because of their large total size, the occurrence lists have to be stored on disk.

With our data structure, a query for a single keyword is similar to a word matching query (see Section 9.2.1). Namely, we find the keyword in the trie and we return the associated occurrence list.

When multiple keywords are given and the desired output is the pages containing *all* the given keywords, we retrieve the occurrence list of each keyword using the trie and return their intersection. To facilitate the intersection computation, each occurrence list should be implemented with a sequence sorted by address or with a dictionary (see, for example, the generic merge computation discussed in Section 4.2).

In addition to the basic task of returning a list of pages containing given keywords, search engines provide an important additional service by *ranking* the pages returned by relevance. Devising fast and accurate ranking algorithms for search engines is a major challenge for computer researchers and electronic commerce companies.

9.3 Text Compression

In this section, we consider another text processing application, *text compression*. In this problem, we are given a string X defined over some alphabet, such as the ASCII or Unicode character sets, and we want to efficiently encode X into a binary string Y (using only the characters 0 and 1). Text compression is useful in any situation where we are communicating over a low-bandwidth channel, such as a slow modem line or wireless connection, and we wish to minimize the time needed to transmit our text. Likewise, text compression is also useful for storing collections of large documents more efficiently, so as to allow for a fixed-capacity storage device to contain as many documents as possible.

The method for text compression explored in this section is the *Huffman code*. Standard encoding schemes, such as the ASCII and Unicode systems, use fixed-length binary strings to encode characters (with 7 bits in the ASCII system and 16 in the Unicode system). A Huffman code, on the other hand, uses a variable-length encoding optimized for the string X . The optimization is based on the use of character *frequencies*, where we have, for each character c , a count $f(c)$ of the number of times c appears in the string X . The Huffman code saves space over fixed-length encoding by using short code-word strings to encode high-frequency characters and long code-word strings to encode low-frequency characters.

To encode the string X , we convert each character in X from its fixed-length code word to its variable-length code word, and we concatenate all these code words in order to produce the encoding Y for X . In order to avoid ambiguities, we insist that no code word in our encoding is a prefix of another code word in our encoding. Such a code is called a *prefix code*, and it simplifies the decoding of Y in order to get back X . (See Figure 9.16.) Even with this restriction, the savings produced by a variable-length prefix code can be significant, particularly if there is a wide variance in character frequencies (as is the case for natural language text in almost every spoken language).

Huffman's algorithm for producing an optimal variable-length prefix code for X is based on the construction of a binary tree T that represents the code. Each node in T , except the root, represents a bit in a code word, with each left child representing a "0" and each right child representing a "1." Each external node v is associated with a specific character, and the code word for that character is defined by the sequence of bits associated with the nodes in the path from the root of T to v . (See Figure 9.16.) Each external node v has a *frequency* $f(v)$, which is simply the frequency in X of the character associated with v . In addition, we give each internal node v in T a frequency, $f(v)$, that is the sum of the frequencies of all the external nodes in the subtree rooted at v .

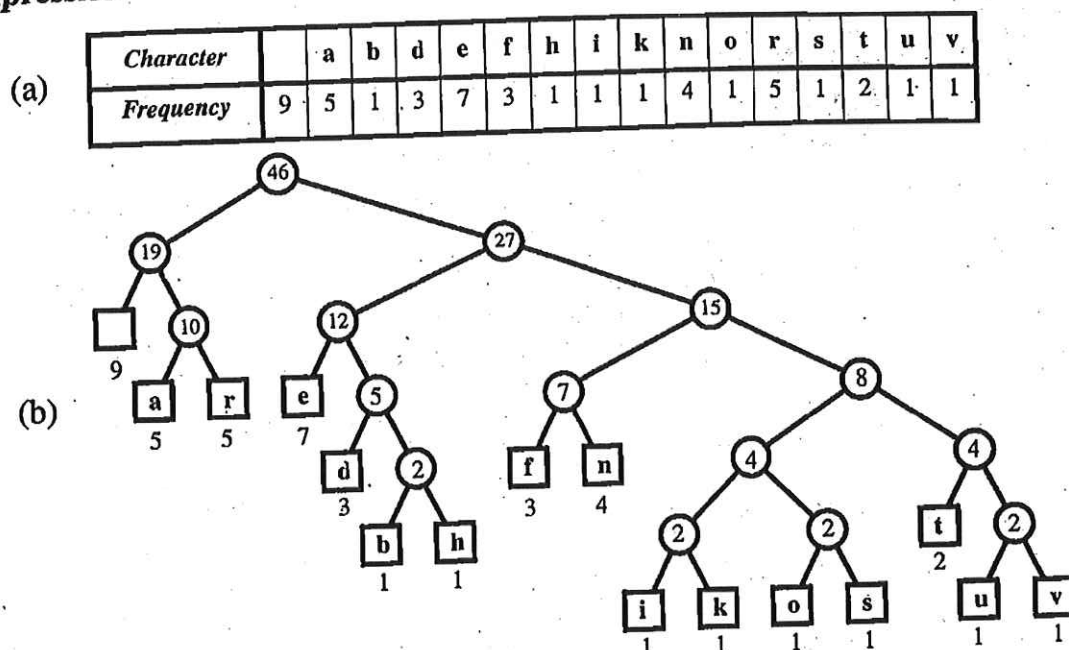


Figure 9.16: An illustration of an example Huffman code for the input string $X = \text{"a fast runner need never be afraid of the dark"}$: (a) frequency of each character of X ; (b) Huffman tree T for string X . The code for a character c is obtained by tracing the path from the root of T to the external node where c is stored, and associating a left child with 0 and a right child with 1. For example, the code for "a" is 010, and the code for "f" is 1100.

9.3.1 The Huffman Coding Algorithm

The Huffman coding algorithm begins with each of the d distinct characters of the string X to encode being the root node of a single-node binary tree. The algorithm proceeds in a series of rounds. In each round, the algorithm takes the two binary trees with the smallest frequencies and merges them into a single binary tree. It repeats this process until only one tree is left. (See Algorithm 9.17.)

Each iteration of the while-loop in Huffman's algorithm can be implemented in $O(\log d)$ time using a priority queue represented with a heap. In addition, each iteration takes two nodes out of Q and adds one in, a process that will be repeated $d - 1$ times before exactly one node is left in Q . Thus, this algorithm runs in $O(n + d \log d)$ time. Although a full justification of this algorithm's correctness is beyond our scope here, we note that its intuition comes from a simple idea—any optimal code can be converted into an optimal code in which the code words for the two lowest-frequency characters, a and b , differ only in their last bit. Repeating the argument for a string with a and b replaced by a character c , gives the following:

Theorem 9.8: Huffman's algorithm constructs an optimal prefix code for a string of length n with d distinct characters in $O(n + d \log d)$ time.

Algorithm Huffman(X):**Input:** String X of length n with d distinct characters**Output:** Coding tree for X Compute the frequency $f(c)$ of each character c of X .Initialize a priority queue Q .**for each** character c in X **do** Create a single-node binary tree T storing c . Insert T into Q with key $f(c)$.**while** $Q.size() > 1$ **do** $f_1 \leftarrow Q.minKey()$ $T_1 \leftarrow Q.removeMin()$ $f_2 \leftarrow Q.minKey()$ $T_2 \leftarrow Q.removeMin()$ Create a new binary tree T with left subtree T_1 and right subtree T_2 . Insert T into Q with key $f_1 + f_2$.**return** tree $Q.removeMin()$

Algorithm 9.17: Huffman coding algorithm.

9.3.2 The Greedy Method Revisited

Huffman's algorithm for building an optimal encoding is an example application of an algorithmic design pattern called the *greedy method*. We recall from Section 5.1 that this design pattern is applied to optimization problems, where we are trying to construct some structure while minimizing or maximizing some property of that structure.

Indeed, the Huffman coding algorithm closely follows the general formula for the greedy method pattern. Namely, in order to solve the given optimization problem using the greedy method, we proceed by a sequence of choices. The sequence starts from a well-understood starting condition, and computes the cost for that initial condition. Finally, we iteratively make additional choices by identifying the decision that achieves the best cost improvement from all of the choices that are currently possible. This approach does not always lead to an optimal solution, but it does indeed find the optimal prefix code when used according to the approach of Huffman's algorithm.

This global optimality for the Huffman coding algorithm is due to the fact that the optimal prefix coding problem possesses the *greedy-choice* property. This is the property that a global optimal condition can be reached by a series of locally optimal choices (that is, choices that are each the current best from among the possibilities available at the time), starting from a well-defined starting condition. In fact, the problem of computing an optimal variable-length prefix code is just one example of a problem that possesses the greedy-choice property.

9.4 Text Similarity Testing

A common text processing problem, which arises in genetics and software engineering, is to test the similarity between two text strings. In a genetics application, the two strings could correspond to two strands of DNA, which could, for example, come from two individuals, who we will consider genetically related if they have a long subsequence common to their respective DNA sequences. Likewise, in a software engineering application, the two strings could come from two versions of source code for the same program, and we may wish to determine which changes were made from one version to the next. In addition, the data gathering systems of search engines, which are called Web *spiders* or *crawlers*, must be able to distinguish between similar Web pages to avoid needless Web page requests. Indeed, determining the similarity between two strings is considered such a common operation that the Unix/Linux operating systems come with a program, called `diff`, for comparing text files.

9.4.1 The Longest Common Subsequence Problem

There are several different ways we can define the similarity between two strings. Even so, we can abstract a simple, yet common, version of this problem using character strings and their subsequences. Given a string X of size n , a *subsequence* of X is any string that is of the form

$$X[i_1]X[i_2]\cdots X[i_k], \quad i_j < i_{j+1} \text{ for } j = 1, \dots, k;$$

that is, it is a sequence of characters that are not necessarily contiguous but are nevertheless taken in order from X . For example, the string `AAAG` is a subsequence of the string `CGATAATTGAGA`. Note that the concept of *subsequence* of a string is different from the one of *substring* of a string, defined in Section 9.1.1.

Problem Definition

The specific text similarity problem we address here is the *longest common subsequence* (LCS) problem. In this problem, we are given two character strings, X of size n and Y of size m , over some alphabet and are asked to find a longest string S that is a subsequence of both X and Y .

One way to solve the longest common subsequence problem is to enumerate all subsequences of X and take the largest one that is also a subsequence of Y . Since each character of X is either in or not in a subsequence, there are potentially 2^n different subsequences of X , each of which requires $O(m)$ time to determine whether it is a subsequence of Y . Thus, the brute-force approach yields an exponential algorithm that runs in $O(2^n m)$ time, which is very inefficient. In this section, we discuss how to use *dynamic programming* (Section 5.3) to solve the longest common subsequence problem much faster than this.

9.4.2 Applying Dynamic Programming to the LCS Problem

We can solve the LCS problem much faster than exponential time using dynamic programming. As mentioned in Section 5.3, one of the key components of the dynamic programming technique is the definition of simple subproblems that satisfy the subproblem optimization and subproblem overlap properties.

Recall that in the LCS problem, we are given two character strings, X and Y , of length n and m , respectively, and are asked to find a longest string S that is a subsequence of both X and Y . Since X and Y are character strings, we have a natural set of indices with which to define subproblems—indices into the strings X and Y . Let us define a subproblem, therefore, as that of computing the length of the longest common subsequence of $X[0..i]$ and $Y[0..j]$, denoted $L[i, j]$.

This definition allows us to rewrite $L[i, j]$ in terms of optimal subproblem solutions. We consider the following two cases. (See Figure 9.18.)

Case 1: $X[i] = Y[j]$

Let $c = X[i] = Y[j]$. We claim that a longest common subsequence of $X[0..i]$ and $Y[0..j]$ ends with c . To prove this claim, let us suppose it is not. There has to be some longest common subsequence $X[i_1]X[i_2] \dots X[i_k]Y[j_1]Y[j_2] \dots Y[j_k]$. If $X[i_k] = c$ or $Y[j_k] = c$, then we get the same sequence by setting $i_k = i$ and $j_k = j$. Alternately, if $X[i_k] \neq c$ and $Y[j_k] \neq c$, then we can get a longer common subsequence by adding c to the end. Thus, a longest common subsequence of $X[0..i]$ and $Y[0..j]$ ends with $c = X[i] = Y[j]$. Therefore, we set

$$L[i, j] = L[i-1, j-1] + 1 \quad \text{if } X[i] = Y[j].$$

Case 2: $X[i] \neq Y[j]$

In this case, we cannot have a common subsequence that includes both $X[i]$ and $Y[j]$. That is, a common subsequence can end with $X[i]$, $Y[j]$, or neither, but not both. Therefore, we set

$$L[i, j] = \max\{L[i-1, j], L[i, j-1]\} \quad \text{if } X[i] \neq Y[j].$$

In order to make Equations 9.1 and 9.2 make sense in the boundary cases where $i = 0$ or $j = 0$, we define $L[i, -1] = 0$ for $i = -1, 0, 1, \dots, n-1$ and $L[-1, j] = 0$ for $j = -1, 0, 1, \dots, m-1$.

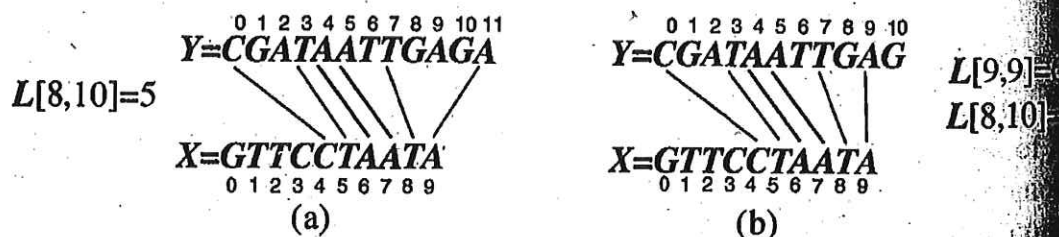


Figure 9.18: The two cases in the definition of $L[i, j]$: (a) $X[i] = Y[j]$; (b) $X[i] \neq Y[j]$.

The LCS Algorithm

The above definition of $L[i, j]$ satisfies subproblem optimization, for we cannot have a longest common subsequence without also having longest common subsequences for the subproblems. Also, it uses subproblem overlap, because a subproblem solution $L[i, j]$ can be used in several other problems (namely, the problems $L[i+1, j]$, $L[i, j+1]$, and $L[i+1, j+1]$).

Turning this definition of $L[i, j]$ into an algorithm is actually quite straightforward. We initialize an $(n+1) \times (m+1)$ array, L , for the boundary cases when $i = 0$ or $j = 0$. Namely, we initialize $L[i, -1] = 0$ for $i = -1, 0, 1, \dots, n-1$ and $L[-1, j] = 0$ for $j = -1, 0, 1, \dots, m-1$. (This is a slight abuse of notation, since in reality, we would have to index the rows and columns of L starting with 0.) Then, we iteratively build up values in L until we have $L[n-1, m-1]$, the length of a longest common subsequence of X and Y . We give a pseudo-code description of how this approach results in a dynamic programming solution to the longest common subsequence (LCS) problem in Algorithm 9.19. Note that the algorithm stores only the $L[i, j]$ values, not the matches.

Algorithm LCS(X, Y):

Input: Strings X and Y with n and m elements, respectively

Output: For $i = 0, \dots, n-1$, $j = 0, \dots, m-1$, the length $L[i, j]$ of a longest common subsequence of $X[0..i]$ and $Y[0..j]$

```

for  $i \leftarrow -1$  to  $n-1$  do
     $L[i, -1] \leftarrow 0$ 
for  $j \leftarrow 0$  to  $m-1$  do
     $L[-1, j] \leftarrow 0$ 
for  $i \leftarrow 0$  to  $n-1$  do
    for  $j \leftarrow 0$  to  $m-1$  do
        if  $X[i] = Y[j]$  then
             $L[i, j] \leftarrow L[i-1, j-1] + 1$ 
        else
             $L[i, j] \leftarrow \max\{L[i-1, j], L[i, j-1]\}$ 
return array  $L$ 

```

Algorithm 9.19: Dynamic programming algorithm for the LCS problem.

Performance

The running time of Algorithm 9.19 is easy to analyze, for it is dominated by two nested for-loops, with the outer one iterating n times and the inner one iterating m times. Since the if-statement and assignment inside the loop each requires $O(1)$ primitive operations, this algorithm runs in $O(nm)$ time. Thus, the dynamic programming technique can be applied to the longest common subsequence problem

to improve significantly over the exponential-time brute-force solution to the problem.

Algorithm LCS (9.19) computes the length of the longest common subsequence (stored in $L[n-1, m-1]$), but not the subsequence itself. As shown in the following theorem, a simple postprocessing step can extract the longest common subsequence from the array L returned by the algorithm.

Theorem 9.9: Given a string X of n characters and a string Y of m characters, we can find the longest common subsequence of X and Y in $O(nm)$ time.

Proof: We have already observed that Algorithm LCS computes the *length* of the longest common subsequence of the input strings X and Y in $O(nm)$ time. Given the table of $L[i, j]$ values, constructing a longest common subsequence is straightforward. One method is to start from $L[n-1, m-1]$ and work back through the table, reconstructing a longest common subsequence from back to front. At any position $L[i, j]$, we determine whether $X[i] = Y[j]$. If this is true, then we take $X[i]$ as the next character of the subsequence (noting that $X[i]$ is *before* the previous character we found, if any), moving next to $L[i-1, j-1]$. If $X[i] \neq Y[j]$, then we move to the larger of $L[i, j-1]$ and $L[i-1, j]$. (See Figure 9.20.) We stop when we reach a boundary entry (with $i = -1$ or $j = -1$). This method constructs a longest common subsequence in $O(n+m)$ additional time.

L	-1	0	1	2	3	4	5	6	7	8	9	10	11
-1	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	1	1	1	1	1	1	1	1	1	1	1
1	0	0	1	1	2	2	2	2	2	2	2	2	2
2	0	0	1	1	2	2	2	3	3	3	3	3	3
3	0	1	1	1	2	2	2	3	3	3	3	3	3
4	0	1	1	1	2	2	2	3	3	3	3	3	3
5	0	1	1	1	2	2	2	3	4	4	4	4	4
6	0	1	1	2	2	3	3	3	4	4	5	5	5
7	0	1	1	2	2	3	4	4	4	4	5	5	6
8	0	1	1	2	3	3	4	5	5	5	5	5	6
9	0	1	1	2	3	4	4	5	5	5	6	6	6

0 1 2 3 4 5 6 7 8 9 10
 $Y = \text{CGATAATTGAG}$
 $X = \text{GTTCTAATA}$
 0 1 2 3 4 5 6 7 8 9

Figure 9.20: Illustration of the algorithm for constructing a longest common subsequence from the array L .

9.5 Exercises

Reinforcement

- R-9.1 How many nonempty prefixes of the string $P = \text{"aaabbaaa"}$ are also suffixes of P ?
- R-9.2 Draw a figure illustrating the comparisons done by the brute-force pattern matching algorithm for the case when the text is "aaabaadaabaaa" and the pattern is "aabaaa" .
- R-9.3 Repeat the previous problem for the BM pattern matching algorithm, not counting the comparisons made to compute the last function.
- R-9.4 Repeat the previous problem for the KMP pattern matching algorithm, not counting the comparisons made to compute the failure function.
- R-9.5 Compute a table representing the last function used in the BM pattern matching algorithm for the pattern string

$\text{"the quick brown fox jumped over a lazy cat"}$

assuming the following alphabet (which starts with the space character):

$$\Sigma = \{ , a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z \}.$$

- R-9.6 Assuming that the characters in alphabet Σ can be enumerated and can index arrays, give an $O(m + |\Sigma|)$ time method for constructing the last function from an m -length pattern string P .
- R-9.7 Compute a table representing the KMP failure function for the pattern string "cgtacgttcgtac" .
- R-9.8 Draw a standard trie for the following set of strings:

$\{\text{abab}, \text{baba}, \text{cccc}, \text{bbaaaa}, \text{caa}, \text{bbaacc}, \text{cbcc}, \text{cbca}\}.$

- R-9.9 Draw a compressed trie for the set of strings given in Exercise R-9.8.
- R-9.10 Draw the compact representation of the suffix trie for the string

"minimize minime" .

- R-9.11 What is the longest prefix of the string "cgtacgttcgtacg" that is also a suffix of this string?

- R-9.12 Draw the frequency table and Huffman tree for the following string:

$\text{"dogs do not spot hot pots or cats"}$.

- R-9.13 Show how to use dynamic programming to compute the longest common subsequence between the two strings "babbabab" and "bbabbbaaab" .

R-9.14 Show the longest common subsequence table L for the two strings

$X = \text{"skullandbones"}$

$Y = \text{"lullabybabies"}$.

What is a longest common subsequence between these strings?

Creativity

- C-9.1 Give an example of a text T of length n and a pattern P of length m that force the brute-force pattern matching algorithm to have a running time that is $\Omega(nm)$.
- C-9.2 Give a justification of why the KMPFailureFunction method (Algorithm 9.1.4) runs in $O(m)$ time on a pattern of length m .
- C-9.3 Show how to modify the KMP string pattern matching algorithm so as to find *every* occurrence of a pattern string P that appears as a substring in T , while still running in $O(n + m)$ time. (Be sure to catch even those matches that overlap.)
- C-9.4 Let T be a text of length n , and let P be a pattern of length m . Describe an $O(n + m)$ -time method for finding the longest prefix of P that is a substring of T .
- C-9.5 Say that a pattern P of length m is a *circular substring* of a text T of length n if there is an index $0 \leq i < m$, such that $P = T[n - m + i..n - 1] + T[0..i - 1]$, that is, if P is a substring of T or P is equal to the concatenation of a suffix of T and a prefix of T . Give an $O(n + m)$ -time algorithm for determining whether P is a circular substring of T .
- C-9.6 The KMP pattern matching algorithm can be modified to run faster on binary strings by redefining the failure function as
- $$f(j) = \text{the largest } k < j \text{ such that } P[0..k-2] \overline{P[k-1]} \text{ is a suffix of } P[1..j],$$
- where $\overline{P[k]}$ denotes the complement of the k th bit of P . Describe how to modify the KMP algorithm to be able to take advantage of this new failure function and also give a method for computing this failure function. Show that this method makes at most n comparisons between the text and the pattern (as opposed to the $2n$ comparisons needed by the standard KMP algorithm given in Section 9.1.4).
- C-9.7 Modify the simplified BM algorithm presented in this chapter using ideas from the KMP algorithm so that it runs in $O(n + m)$ time.
- C-9.8 Show how to perform prefix matching queries using a suffix trie.
- C-9.9 Give an efficient algorithm for deleting a string from a standard trie and analyze its running time.
- C-9.10 Give an efficient algorithm for deleting a string from a compressed trie and analyze its running time.
- C-9.11 Describe an algorithm for constructing the compact representation of a suffix trie and analyze its running time.
- C-9.12 Let T be a text string of length n . Describe an $O(n)$ -time method for finding the longest prefix of T that is a substring of the reversal of T .

- C-9.13 Describe an efficient algorithm to find the longest palindrome that is a suffix of a string T of length n . Recall that a *palindrome* is a string that is equal to its reversal. What is the running time of your method?
- C-9.14 Given a sequence $S = (x_0, x_1, x_2, \dots, x_{n-1})$ of numbers, describe an $O(n^2)$ -time algorithm for finding a longest subsequence $T = (x_{i_0}, x_{i_1}, x_{i_2}, \dots, x_{i_{k-1}})$ of numbers, such that $i_j < i_{j+1}$ and $x_{i_j} > x_{i_{j+1}}$. That is, T is a longest decreasing subsequence of S .
- C-9.15 Describe an $O(n \log n)$ -time algorithm for the previous problem.
- C-9.16 Show that a sequence of n distinct numbers contains a decreasing or increasing subsequence of size at least $\lfloor \sqrt{n} \rfloor$.
- C-9.17 Define the *edit distance* between two strings X and Y of length n and m , respectively, to be the number of edits that it takes to change X into Y . An edit consists of a character insertion, a character deletion, or a character replacement. For example, the strings "algorithm" and "rhythm" have edit distance 6. Design an $O(nm)$ -time algorithm for computing the edit distance between X and Y .
- C-9.18 Let A , B , and C be three length- n character strings taken over the same constant-sized alphabet Σ . Design an $O(n^3)$ -time algorithm for finding a longest substring that is common to all three of A , B , and C .
- C-9.19 Consider the pattern matching algorithm that always returns "no" when asked to determine whether a pattern P of length m is contained in a text T of length n , with both taken over the same alphabet of size d for some constant $d > 1$. Give a big-Oh characterization of the probability that this simple algorithm incorrectly determines whether or not P is a substring in T , assuming that all possible pattern strings of length m are equally likely? Your bound must be $o(1)$.
Hint: This is a remarkably accurate algorithm when m is large.
- C-9.20 Suppose A , B , and C are three integer arrays representing the ASCII or Unicode values of three character strings, each of size n . Given an arbitrary integer x , design an $O(n^2 \log n)$ -time algorithm to determine if there exist numbers $a \in A$, $b \in B$, and $c \in C$, such that $x = a + b + c$.
- C-9.21 Give an $O(n^2)$ -time algorithm for the previous problem.
- C-9.22 Suppose each character c in a constant-sized alphabet Σ has an integer worth, $w(c)$. Two players are going to play a game, given a string P of length $2n$ taken over Σ . In each turn, a player must select and remove either the first or last character in P , reducing its length by one. The goal of each player is to maximize the total worth of all his or her selected characters. Give an $O(n^2)$ -time algorithm for computing the optimal strategy for the first player.
Hint: Use dynamic programming.
- C-9.23 Design an $O(n)$ -time non-losing strategy for the first player in the game of the previous exercise. Your strategy does not have to be optimal, but it should be guaranteed to end in a tie or better for the first player.

